

Machine Learning

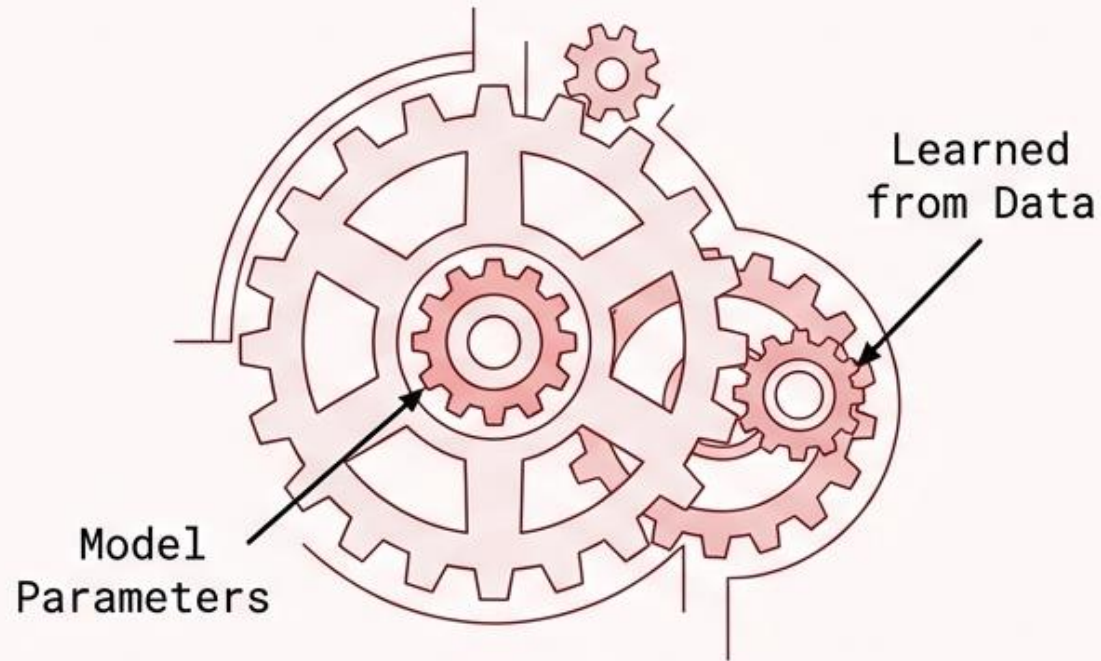
Week 11 Lecture

Dr. Muhammad Ahmed Raza

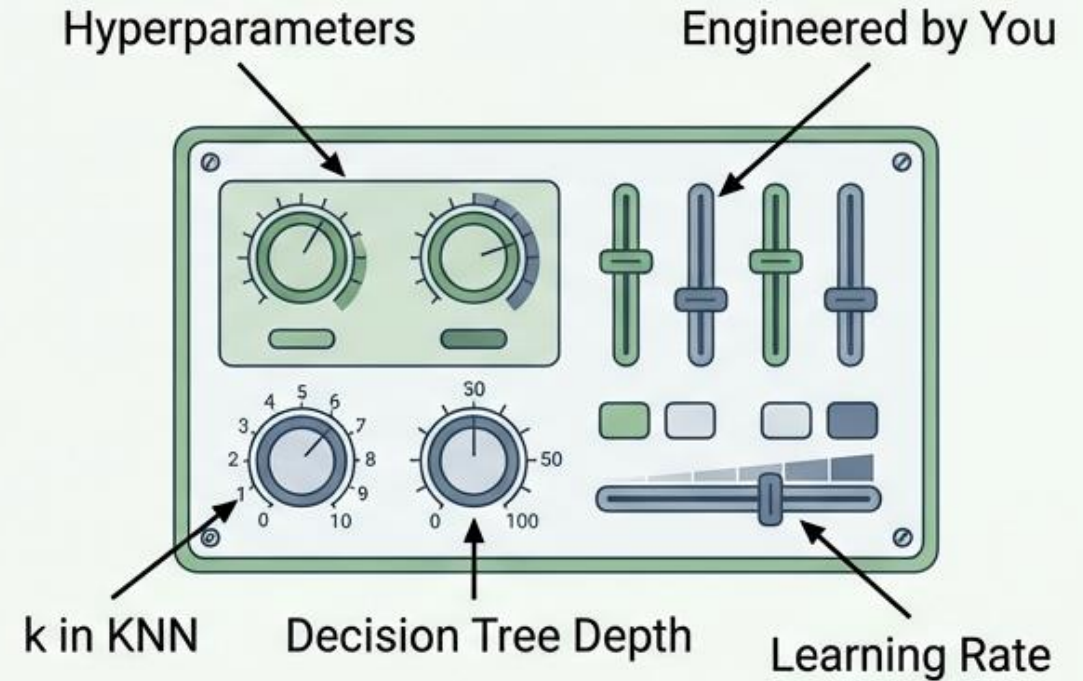
Contents

- Model Optimization & Feature Engineering
- Grid / Random / Bayesian Search (Optuna)
- Cross-Validation
- Target Encoding
- Leakage
- Cyclical Features

The Naïve Engine



The Robust Engine

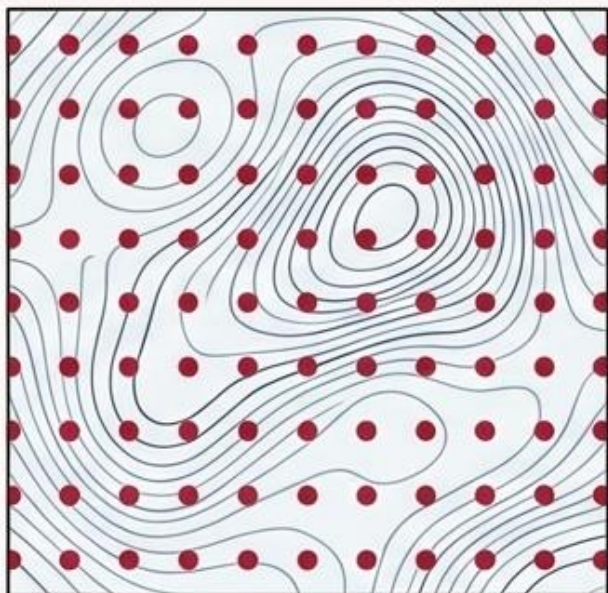


Model optimization is finding the exact control dial configurations that maximize performance. Manual tuning is biased, slow, and non-reproducible. We need automated search.

Tuning

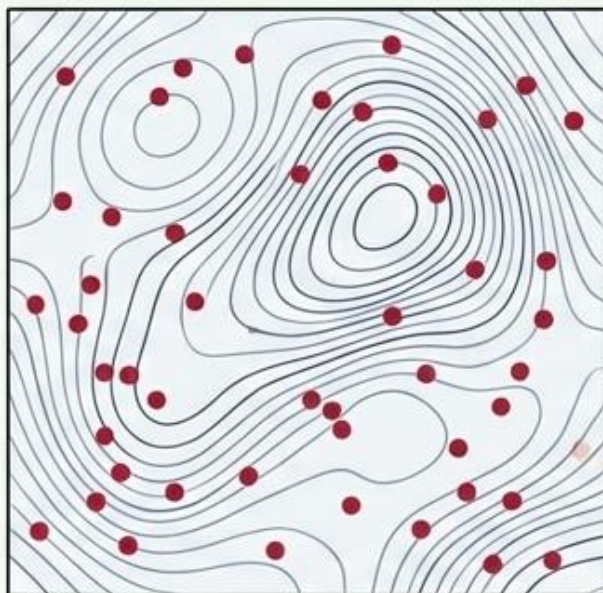
- Manual Tuning:
 - Manual tuning is: ❌ Time-consuming ❌ Biased ❌ Not reproducible
- Automated Tuning:
 - Search:
 - Grid search, random search, Bayesian optimization
 - Using hyperparameter optimization libraries:
 - optuna, Wandb, Hyperopt, Etc.

Grid Search



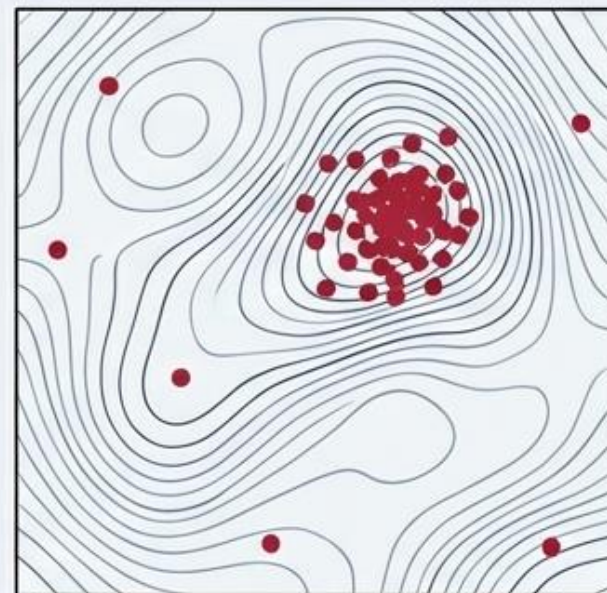
Exhaustive but scales poorly.
Misses peaks between the
grid lines.

Random Search



Faster and better for high
dimensions. Proves not all
hyperparameters matter equally.

Bayesian Optimization



Learns from previous trials.
Balances exploration with
exploitation.

Automated search methods iterate over hyperparameter configurations to find the optimal settings, improving model performance and reproducibility.

Bayesian Optimization

- Idea:
 - Learn from previous trials to choose better hyperparameters.
 - Instead of random guessing:
 - Build a probabilistic model
 - Focus on promising regions
- Algorithm:
 - Try initial random points
 - Fit surrogate model
 - Select next best candidate
 - Repeat

Optuna

- To use Optuna, you only need to know three things:
 - Objective Function:
 - A function that takes a "trial", suggests parameters, and returns a score (like accuracy or error).
 - Trial:
 - A single execution of the objective function.
 - Study:
 - A session that manages a set of trials to find the optimal value.
- Optuna doesn't guess:
 - uses algorithms like TPE (Tree-structured Parzen Estimator) to model the relationship between your parameters and the score.

Optuna in Action

```
import optuna
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split

# 1. Setup data
X, y = make_regression(n_samples=1000, n_features=10, noise=0.1, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2)

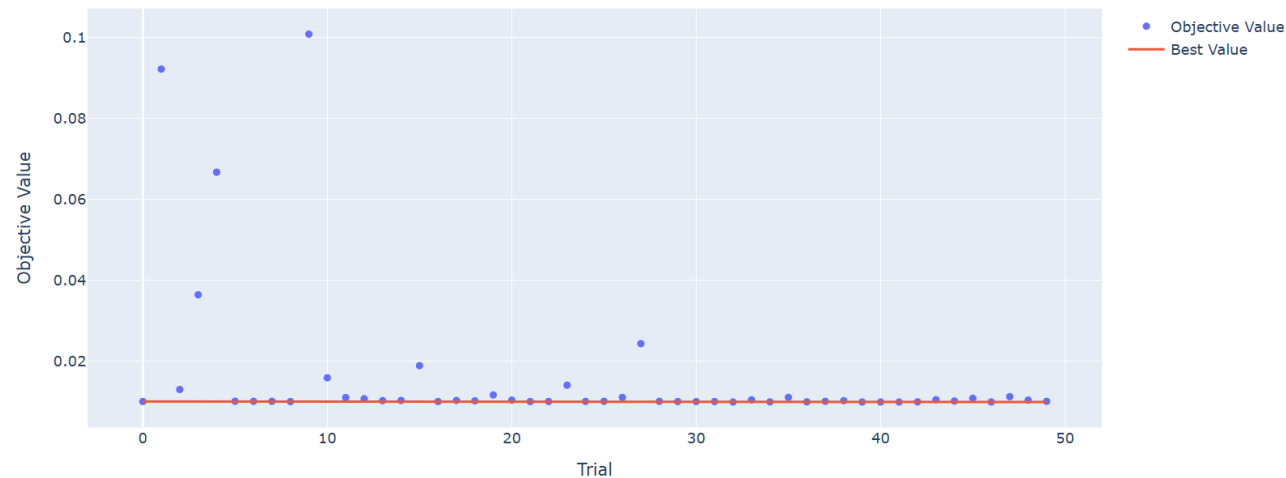
def objective(trial):
    # 2. Suggest a learning rate (on a log scale is usually better for LR)
    lr = trial.suggest_float("lr", 1e-5, 1e-1, log=True)

    # 3. Train the model
    model = SGDRegressor(learning_rate='constant', eta0=lr, random_state=42)
    model.fit(X_train, y_train)

    # 4. Evaluate
    preds = model.predict(X_val)
    error = mean_squared_error(y_val, preds)
    return error

# 5. Run the study
study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=50)

print(f"Best Learning Rate: {study.best_params['lr']}")
```



Optuna (Exploration vs. Exploitation)

- Instead of just picking one "best" area, it maintains two separate probability distributions:
 - $l(x)$ (The "Good" Distribution): Formed by the best trials.
 - $g(x)$ (The "Bad" Distribution): Formed by everything else.
 - To pick the next point, it looks for the value that is highly likely in the "good" distribution but very unlikely in the "bad" distribution.
 - Even late in the study, if there is a large "empty" area in your search space that hasn't been tested, the "bad" distribution doesn't have data there yet.
 - This makes that far-away point technically "interesting" to the algorithm, so it takes a gamble and tries it.

	Grid Search	Random Search	Bayesian (Optuna)
Search Space Coverage	Full / Exhaustive	Partial / Scattered	Targeted / Focused
Execution Speed	Slow	Fast	Fast
Intelligence Mode	Static ❌	Static ⚠️	Adaptive ✅
Optimal Use Case	Small search spaces	General use and baselines	Expensive models where efficiency is critical

The Naïve Approach: Single Split



Single Split = High variance.
Unstable results. Model overfits
to one specific slice of data.

The Robust Approach: K-Fold Cross-Validation

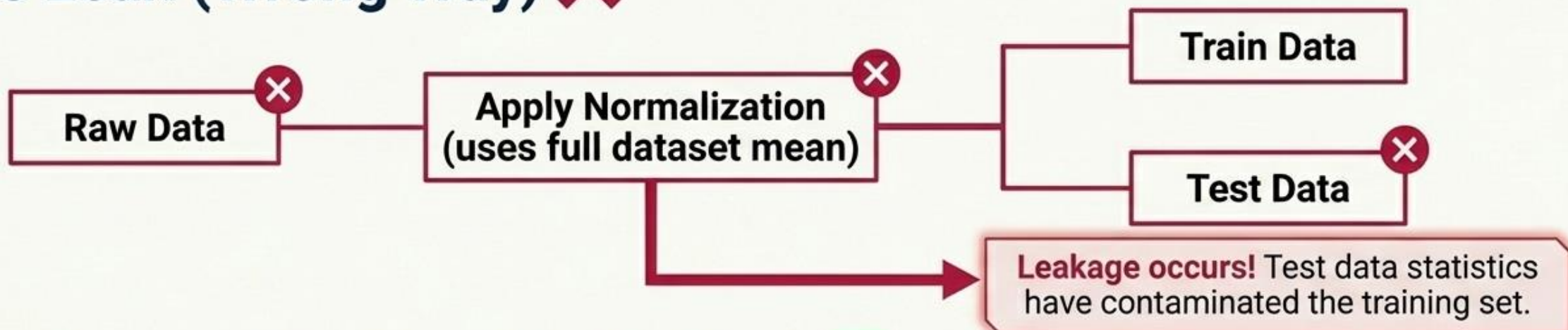
Train	Train	Train	Train	Test
Train	Train	Train	Test	Train
Train	Train	Test	Train	Train
Train	Test	Train	Train	Train
Test	Train	Train	Train	Train

Final Score = Average of K folds. Ensures a reliable estimate by utilizing 100% of the data for both training and validation across iterations.

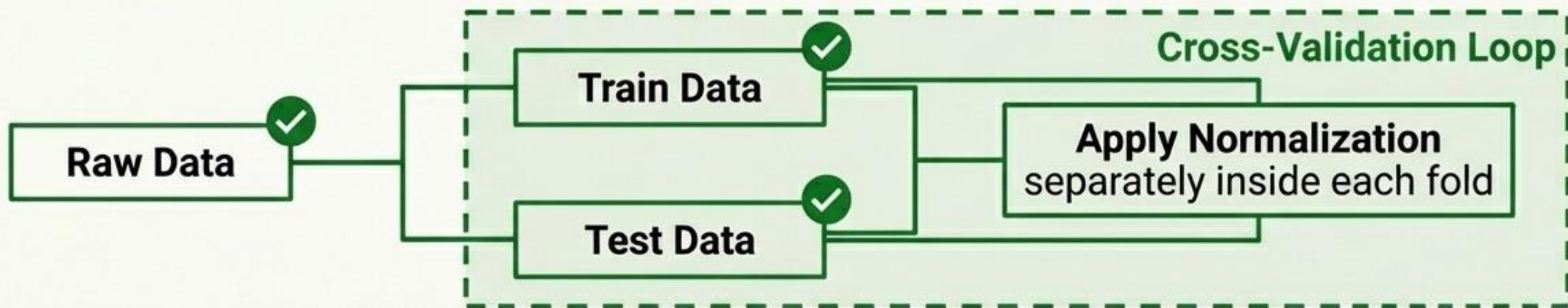
Data Leakage

- Definition:
 - Data leakage is the inclusion of information about the target variable in the training data, which will not be available when the model is used for prediction.
- Types of Leakage
 - Train-Test Contamination:
 - Occurs when information from the test or validation set leaks into the training dataset during preprocessing, such as filling in missing values using data from the entire dataset rather than just the training subset.
 - Target Leakage:
 - Occurs when predictors include data that will not be available at the time of prediction (e.g., using patient health data from after a diagnosis to predict the diagnosis itself).

The Leak (Wrong Way) ✖



The Secure Pipeline (Right Way) ✔



ALWAYS apply preprocessing **inside each fold** of your cross-validation loop.

Feature Engineering

- Categorical Variables
 - Binary features (yes/no, etc.)
 - Nominal Features (no order based categories, i.e., Lahore, Multan etc)
 - Ordinal Features (ordered categories, small/medium/large)
 - Cardinality:
 - This refers to the number of unique categories within a feature
- Cyclical Features (days, times, seasons, etc.)

Categorical Variables Are Hard

- Numerical models expect numbers, but categories are labels, not quantities.

Customer	City	Bought?
A	Lahore	1
B	Karachi	0
C	Multan	1

- After encoding:

City	Encoded
Lahore	1
Karachi	2
Multan	3

- Model interprets: Karachi (2) > Lahore (1) -> false ordinal relationships

One hot Encoding

City_Lahore	City_Karachi	City_Multan
1	0	0
0	1	0
0	0	1

- Advantages
 - No false ordering
 - Works well with most models
- Downsides:
 - High dimensionality:
 - 1000 cities → 1000 columns
 - Sparsity
 - Mostly zeros
 - Memory + computation cost

Target Encoding

- Replace Category with mean target value
 - Feature city, and bought is labels/target

City	Bought
Lahore	1
Lahore	0
Karachi	0
Karachi	0
Multan	1

- Compute:
 - Lahore $\rightarrow (1+0)/2 = 0.5$
 - Karachi $\rightarrow (0+0)/2 = 0.0$
 - Multan $\rightarrow 1/1 = 1.0$

One-Hot Encoding Issue

[illegible]

Target Encoding

0.8
0.6
0.9
0.5
0.7
0.8
0.5
0.7
0.6
0.7
0.4
0.6
0.9
0.6
0.5
0.5
0.7
0.7
0.7

	City	Target
1	Lahore	0.8
2	Karachi	0.6

 NotebookLM



The Danger: Overfitting.

If data is sparse, the model **memorizes categories** instead of learning patterns.

Especially dangerous with small datasets.

The Engineering Solutions



Smoothing Factor: Blends local category average with overall dataset average.



CV Application: Compute encoding **ONLY** on the training fold, never on the validation fold.

Why this works

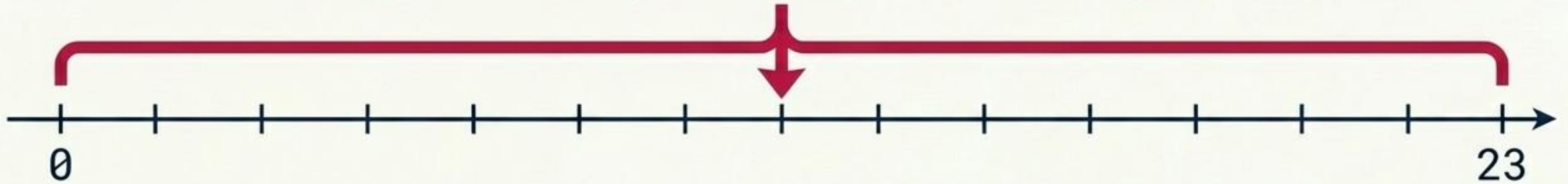
- Captures relationship with target
- Reduces dimensionality
- Works well for high-cardinality features
- Beware  :
 - If you compute using full data:
 - You are leaking the target into features

Summary (Categorical Variables)

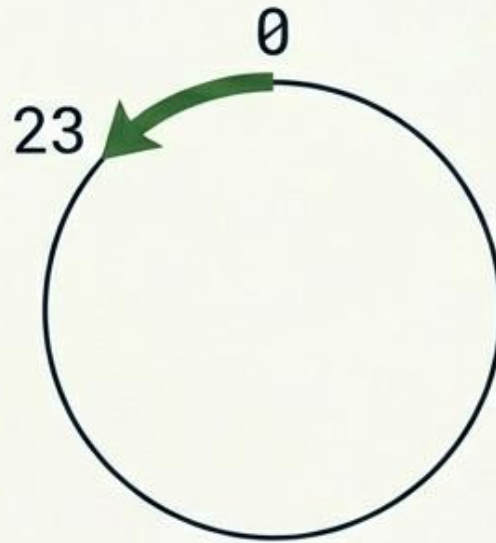
Method	When to Use
Label Encoding	Only for ordered categories (ordinal features)
One-Hot	Low cardinality
Target Encoding	High cardinality

The Model's Flawed View (Linear)

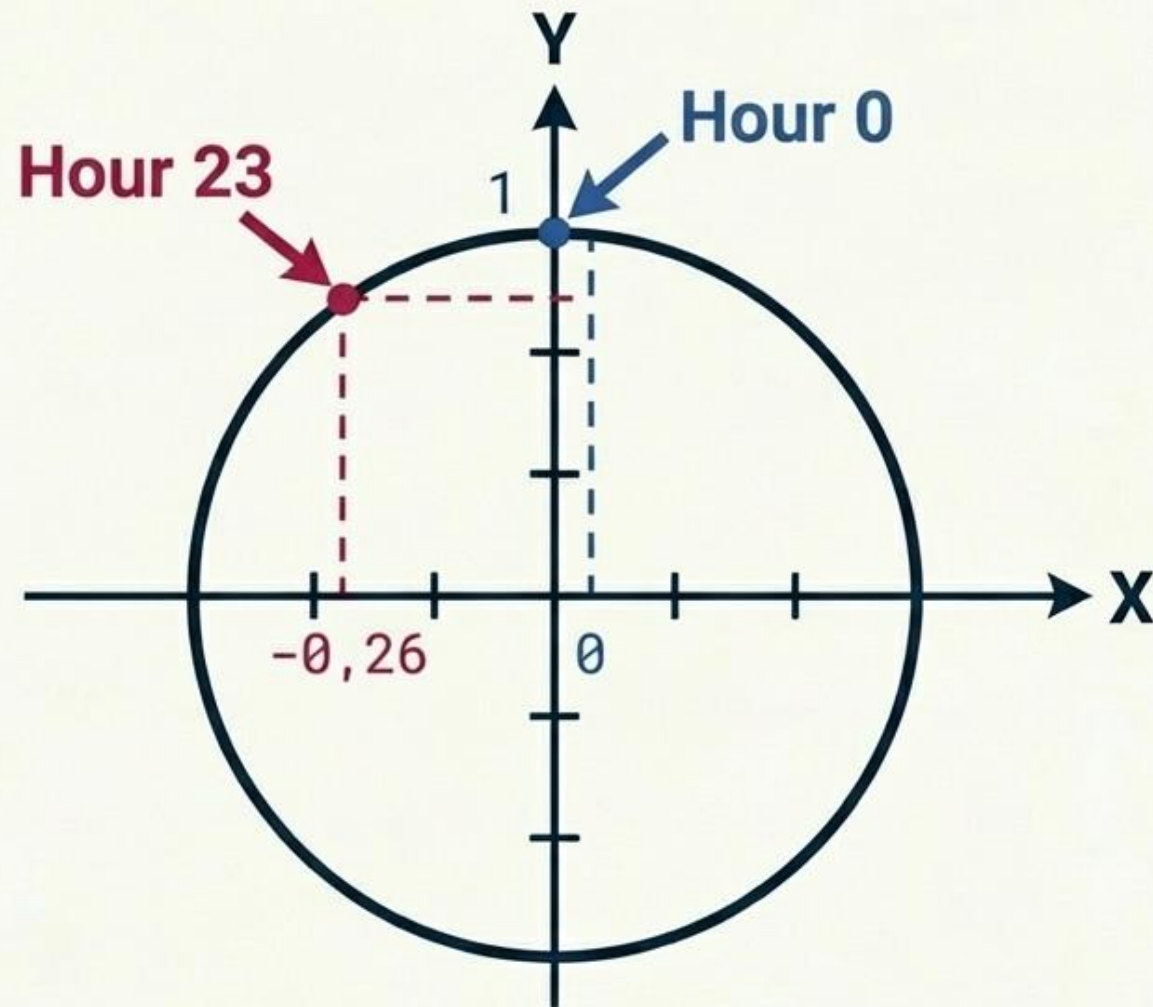
To a naive model, hour 0 and hour 23 are numeric opposites separated by a massive gap.



Reality (Cyclical)



In reality, they are adjacent neighbors. We need to map linear time onto a 2D cyclical space.



Transform Formulas

$$x_{\sin} = \sin\left(\frac{2\pi x}{T}\right)$$

$$x_{\cos} = \cos\left(\frac{2\pi x}{T}\right)$$

Hour 23 Data

With $T = 24$

$$\sin \approx -0.26$$

$$\cos \approx 0.96$$

Hour 0 Data

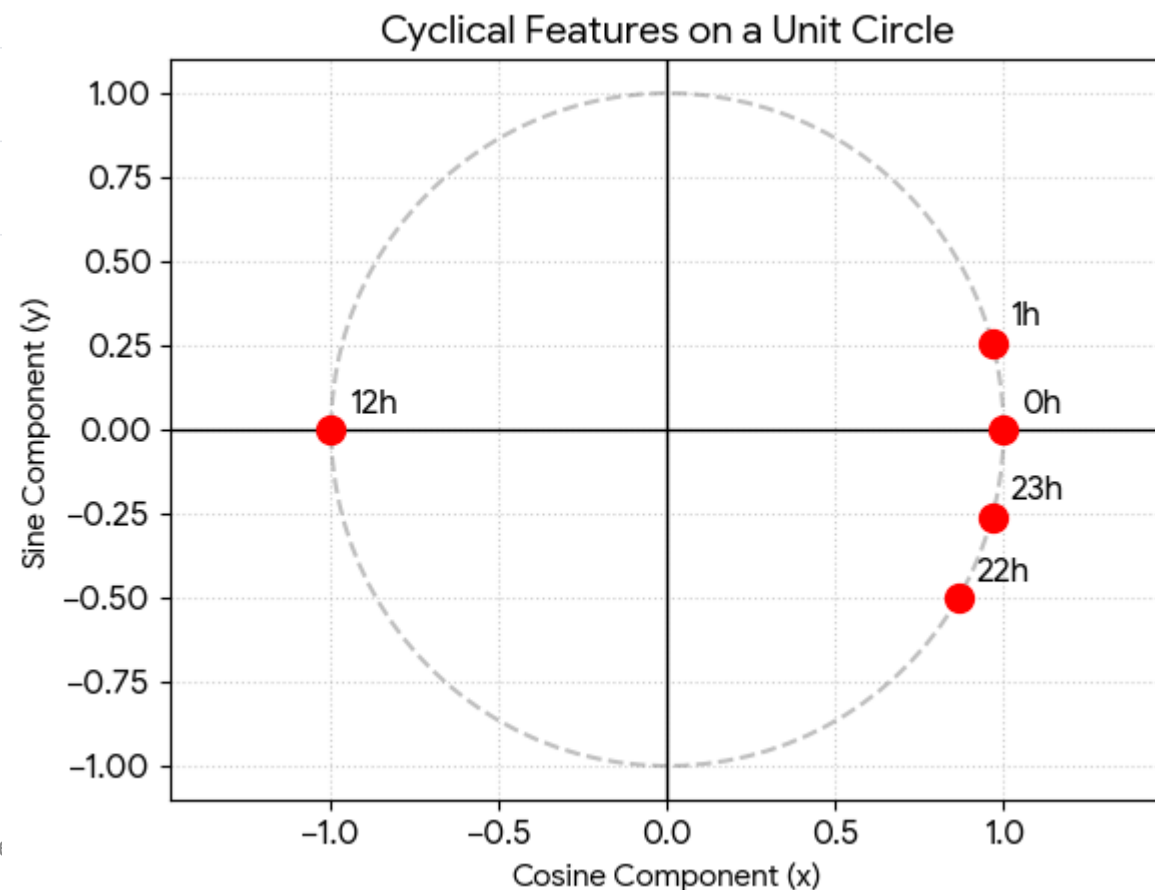
$$\sin = 0$$

$$\cos = 1$$

The transformed coordinates are numerically close across two dimensions, mathematically preserving the cyclic nature for time of day, day of week, or seasonal patterns.

Cyclical Features Example

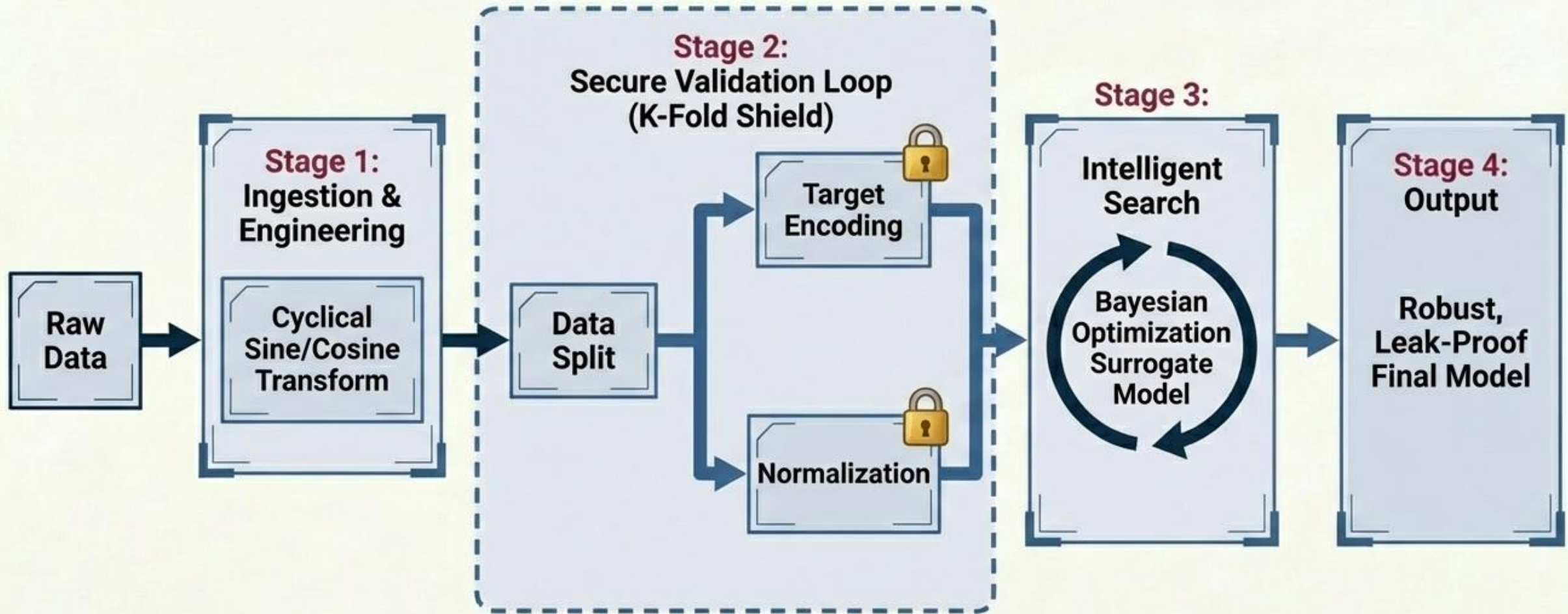
Hour	Time Context	Sine (sin)	Cosine (cos)
22	10:00 PM	-0.500	0.866
23	11:00 PM	-0.259	0.966
0	Midnight	0.000	1.000
1	01:00 AM	0.259	0.966
12	Noon	0.000	-1.000



When to Use Cyclical Encoding

- Use when:
 - Feature wraps around
 - Distance is circular
- Examples:
 - Time, Angles, Seasons
- What does this fix:
 - This fixes the continuity bug
- When NOT to Use
 - No cyclic nature
 - Categories (e.g., city names)

The Blueprint of a Robust ML Pipeline



Engineering Checklist & Summary

Production Standards

- ✓ Use scikit-learn Pipelines to strictly enforce execution order.
- ✓ Isolate all preprocessing logic entirely inside CV folds.
- ✓ Prefer Bayesian or Random Search over Grid brute-force.
- ✓ Apply mathematical smoothing to Target Encoded variables.

Critical Vulnerabilities

- ✗ Encoding categorical variables before executing data splits.
- ✗ Leaking test set statistics (like the global mean) into the training environment.
- ✗ Feeding cyclical time features into models as raw linear integers.